PRODUCTION-AWARE CREDIT RISK SYSTEM

Temporal Leakage Control • MLflow Tracking • API Deployment

Real-world transaction data • 3,194 customers • End-to-end ML system

The Real Problem

Credit scoring fails thin-file customers

Traditional bureau data is sparse or unavailable for many applicants.

- No traditional credit history - Limited underwriting signals - High uncertainty in lending

Can transaction behavior predict risk?

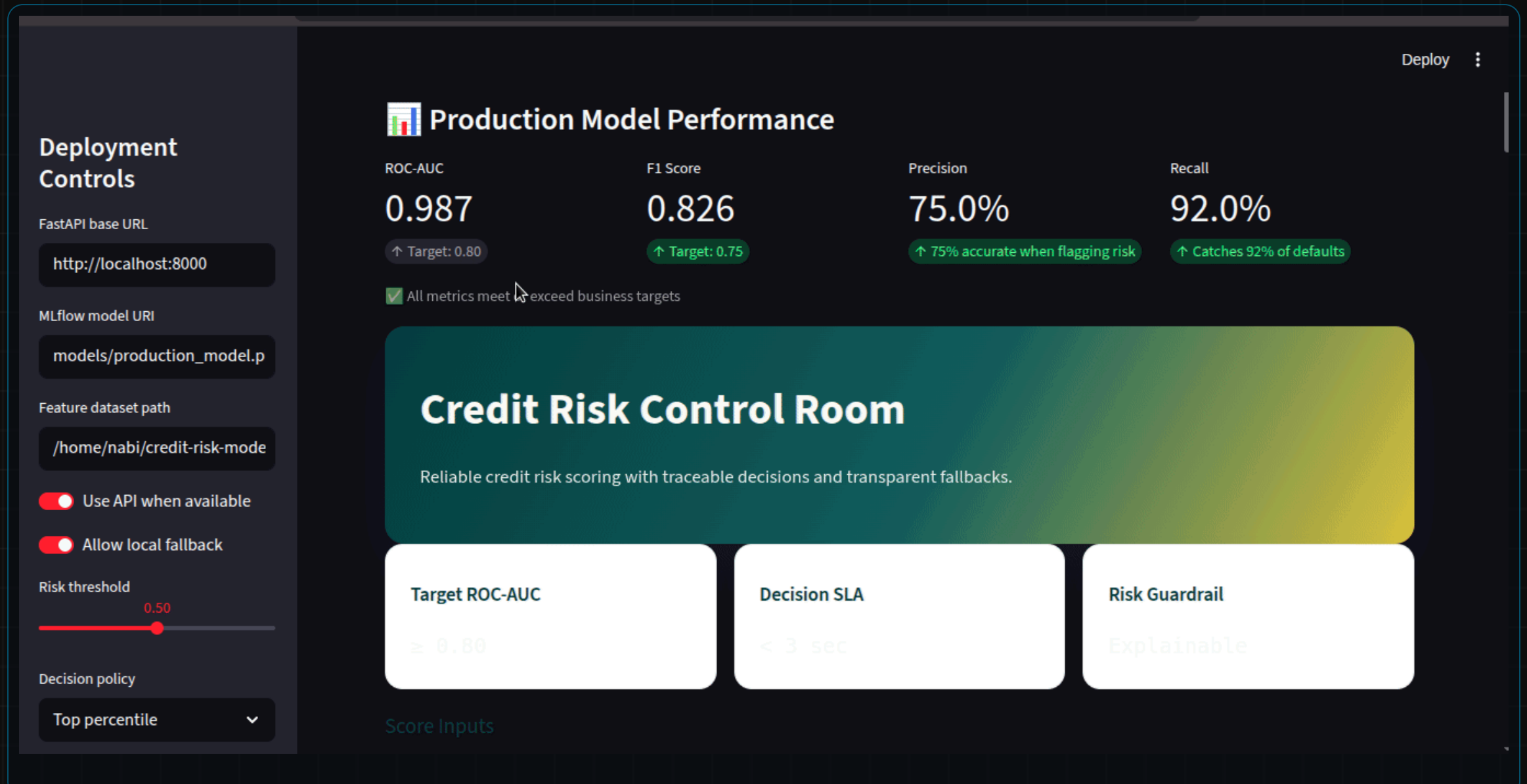
Data → Intelligence Pipeline

```
Raw Transactions
      ↓
Feature Engineering
      ↓
Customer Profiles
      ↓
Leakage-Controlled Risk Labels
      ↓
Production Model
```

3,194	59	95k+
Customers	Engineered Features	Transactions analyzed

Aggregates (sum/avg/count/std) • Datetime behavior • RFM-based risk structure

Dashboard Experience



When Metrics Lie

The Model Was Too Good

ROC-AUC ≈ 1.0

That result looked impressive — and suspicious.

Root Cause: Data Leakage

Feature Window

used for training

```
[==== PAST+FUTURE SIGNAL ====]
                OVERLAP
```

Target Window

used for labeling

```
OVERLAP
[==== SAME PERIOD ====]
```

Leakage: target was influenced by the same period used to construct features.

Temporal Split Fix (Production Design)

```
Past ------------------| cutoff_date |------------------ Future
   build features only                      build target only
```

Strict chronology enforced

No future signal in features

Realistic out-of-time behavior

Performance remained strong even after leakage removal.

Model Strategy & Selection

Logistic Regression

GridSearchCV tuning

Random Forest

RandomizedSearchCV tuning

Cross-validation (cv=3) • RandomOverSampler • Hyperparameter search

Final production choice: Random Forest

Model Evidence (Real Plots)

```
(kaim week1) nabi@nabi-HP-EliteBook-820-G4:~/credit-risk-model$ python scripts/check metrics.py
-
Run ID: 3e1988bf82b244ed9e027c63c6f2e779
f1: 0.8263
accuracy: 0.9613
roc_auc: 0.9869
recall: 0.9200
precision: 0.7500
✅ Model meets criteria!
-
Run ID: f4bee926d2ea49c18c0763e71c6a11ff
f1: 0.8263
accuracy: 0.9613
roc_auc: 0.9869
recall: 0.9200
precision: 0.7500
✅ Model meets criteria!
-
Run ID: 30fa9b4863d54a06ac6b6c5736838380
f1: 0.8156
accuracy: 0.9559
roc_auc: 0.9860
recall: 0.9733
precision: 0.7019
✅ Model meets criteria!
-
Run ID: c4eb4975e96b43c6a330f9994820236e
f1: 0.8156
accuracy: 0.9559
```

Final Performance After Leakage Fix

ROC-AUC

0.9830

Accuracy 0.9446

Precision 0.8686

Recall 0.7810

F1-score 0.8225

Engineering & MLOps Stack

MLflow

experiments
metrics
model artifacts

Reproducibility

requirements.txt
structured src/
deterministic seeds

Deployment

FastAPI /predict
Streamlit UI
JSON inference

Designed for production behavior, observability, and reliable decision support.

API Snippet (Production-Ready)

```python
# src/api/main.py
@app.post("/predict")
def predict(payload: PredictionRequest):
    probs = predict_instances(
        model,
        payload.instances,
        feature_names,
    )
    return {
      "risk_probabilities": probs
    }
```

```
POST /predict
{
  "instances": [
    {
      "total_amount": 0.42,
      "avg_amount": -0.11,
      "txn_count": 0.63,
      "std_amount": 0.08
    }
  ]
}
→ 200 OK

{"risk_probabilities": [0.83]}
```

Health endpoint available: GET /health

End-to-End Runtime Architecture

```
User
  ↓
Streamlit Dashboard
  ↓
FastAPI Inference Layer
  ↓
Production Model (.pkl)
  ↓
Risk Probability Output
```

Primary mode: API scoring

Fallback mode: local model inference

Lessons That Matter

- High accuracy can hide leakage
- Time-aware pipelines are non-negotiable
- Production quality starts at data design
- Engineering discipline > leaderboard metrics

What ships today

Temporal leakage controls
API + dashboard inference
Reproducible training pipeline

What comes next

Out-of-time monitoring
Automated retraining triggers
Containerized cloud deployment

Built to make risk decisions trustworthy in real operations.